

Kill the Quote Spreadsheet

Product & design note · AI Procurement Analyst prototype

The goal

A buyer sends an RFQ for 30 line items to five suppliers. Nine days later the replies arrive as a spreadsheet, a PDF, a Word document, a plain email and a photograph. Three days go into retyping them into Excel. A fourth goes into one question from the VP. We set out to delete that week.

The product bet

The hard part is not collecting quotes, and it is not reading documents. It is that **every number arrives with different conditions attached** — a minimum order, a price basis, a validity window, a currency, a certificate that may or may not exist. The spreadsheet is slow because a human is manually deciding what is comparable with what. **That judgement is the product.**

What we built



Every field carries its provenance. Five document readers feed one AI extraction pass per reply, then deterministic line matching and price normalisation. The buyer interrogates the result in plain English, decides per line, and the decision becomes supplier letters and a structured order handoff with a full audit trail.

Constraints that shaped it

The model must not become the database	Claude turns a question into a structured query and describes the result. Retrieval, filtering, ranking and totals are Python over SQLite.	The analyst can misread what you meant and still cannot be wrong about the number.
Suppliers reply however they like	No template is ever sent. The system absorbs five formats, including a phone photo.	Five companies don't have to change how they work.
Uncertainty is shown, not resolved	A price we cannot reduce to a comparable figure is held out rather than estimated. No fallback exchange rate exists; "\$" is deliberately unmapped.	The buyer sometimes sees a gap. That is the honest answer.
Judgement beats scoring	We were asked for a weighted best-value score and deliberately didn't build one.	A score of 87.3 hides why one supplier won. A named price and a stated rule can be argued with.
No API key	The app shells out to the Claude Code CLI on the user's own subscription.	Runs anywhere with one sign-in, but CLI latency pushed the seven most useful analyst questions to be pre-built and instant.

Trust by design

- **Evidence, or it didn't happen.** A value must trace to a verbatim span in the source or it is downgraded and flagged. The original document is one click from every price.
- **Absence is never zero.** *Not quoted*, *no response*, *unresolved* and a price are four different states.
- **A claimed certificate is not a verified one.** It counts as verified only when the certificate itself is among the documents received — one of six suppliers in the demo data.
- **Supplier text is untrusted input.** Text shaped like an instruction to the AI is dropped before it reaches a prompt, and the buyer is told.
- **One supplier per letter, structurally.** The facts handed to the model hold one supplier and no numeric field; every draft — and every buyer edit — is re-checked against that supplier's own data before it can be sent.
- **The buyer decides.** Overrides require a reason. The audit trail records every state change with what it replaced.

Evaluation

565 automated tests, no network and no model calls, about 30 seconds. They cover the CLI boundary and its failure modes, the Phase 1 trust guards, document extraction, normalisation and line matching, the comparison dataset, analyst calculations and guards, award seeding, validation and execution — plus one end-to-end test that runs a single RFQ through every phase.

Live smoke scripts run the real model against the real fixtures for each phase. The award script fingerprints the upstream tables before and after to prove an award changes nothing it should not, and includes a deliberate cross-supplier leak attempt.

A built-in extraction playground is the answer to “is any of this hardcoded?”. Type or paste any supplier reply, attach any file, and watch the same pipeline read it live — line items, terms, what it could not determine, and the span each value came from. Nothing is saved unless you keep it.

Deliberately ugly demo data: a partial response covering 25 of 30 lines; a revision superseding an earlier PDF; a per-1,000 price basis; a footnote discount with a condition; a minimum order above the line quantity; a self-contradicting lead time; three currencies plus one unnamed and held out; positional line references; an angled photograph; a supplier who never replied; and a prompt-injection line inside a supplier email.

What we refused to build

- **The email loop — outreach, replies, and pulling attachments out of an inbox.** This was the biggest deliberate omission. Wiring a mailbox is a solved, unremarkable problem, and it would have consumed the time that went into the part that is actually hard: deciding what the system should refuse to say. Sending is simulated and labelled as such on every screen — no message leaves the machine. **Note that reading attachments is fully real;** only the transport is stubbed. *Next: an inbound mailbox watcher, feeding the pipeline that already exists.*
- **ERP and PO submission, payment, invoice matching, goods receipt.** The handoff is a downloadable document of record, already structured for it.
- **Supplier portal, logins, permissions.** Forcing suppliers into our UI is the exact behaviour we set out to avoid.
- **Splitting one line across suppliers.** Unrepresentable by design — a unique constraint in the schema holds that line.
- **OCR for scanned PDFs.** A PDF with no text layer is reported unsupported rather than guessed at.

What we learned

Extraction was not the hard part; it worked early. Almost all the difficulty was in deciding **what the system should refuse to say** — and the sharpest lesson came from our own screen. An early version of the award page recommended a supplier and then blocked the award because that supplier's certificate was unverified. The product was arguing with itself. The fix was not technical: by the time a buyer reaches the award screen they have already done the reviewing, so the system should report rather than gate.

The more interesting problem is one layer down. Everyone frames this as document extraction. It isn't. Procurement comparability is a *judgement* — minimum order, price basis, validity, certification — and today that judgement lives only in a buyer's head, which is why it cannot be audited, delegated or checked six months later. Making it explicit and arguable is worth more than reading another PDF format. The most valuable moment in our demo is not the comparison table; it is watching one question — *cheapest per line, but only among suppliers who cleared quality* — reduce a 30-line answer to 11, because only one supplier's certificate is actually on file.

What we would build next

Inbound email ingestion. Buyer-tunable comparability rules — today the quality bar is one toggle, and it wants to be a small, named, reusable policy. And a diff view across supplier revisions, since the revision chain is already stored and currently unexploited.

AI should remove the spreadsheet work, not the buyer from the decision — and where the data will not support an answer, the honest move is to show the gap.